

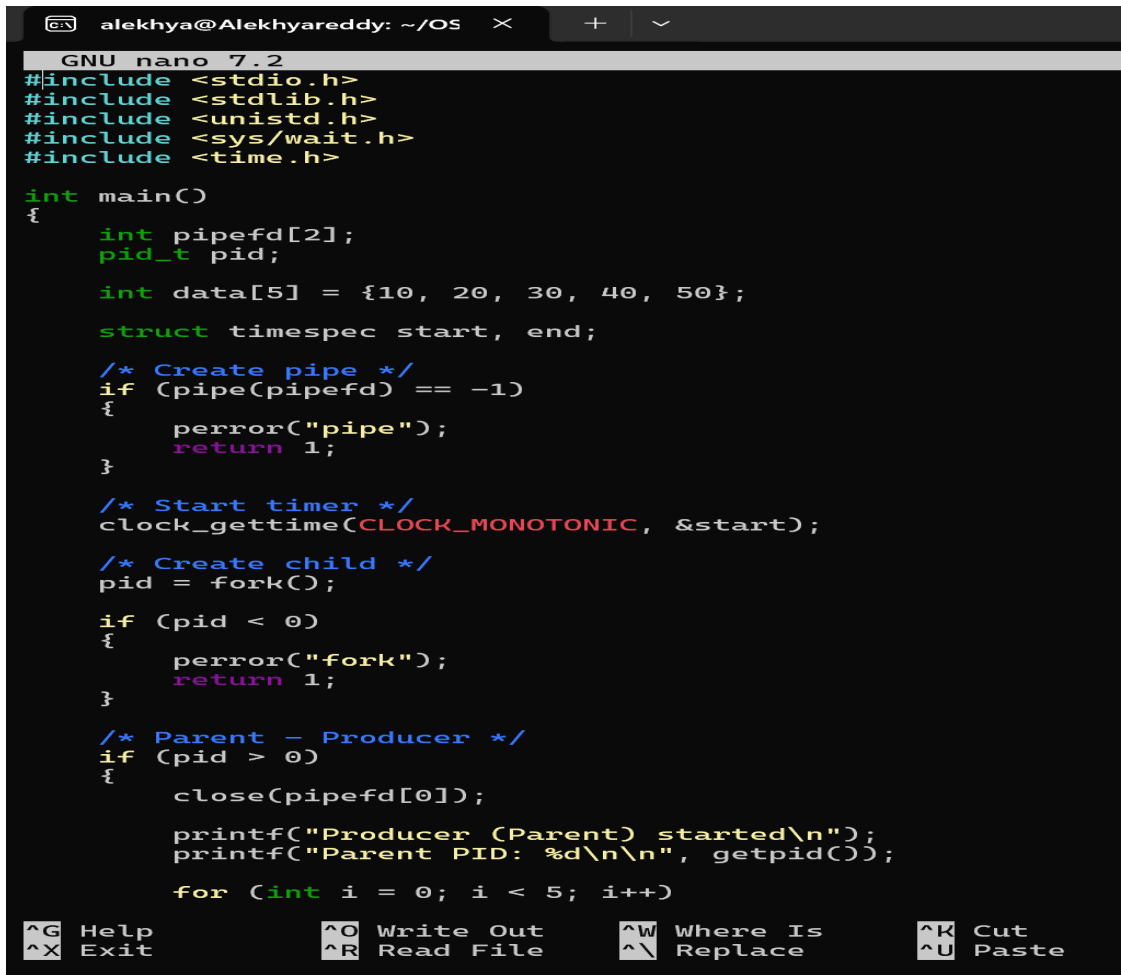
WEEK - 5 PRACTICAL

1 . Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

AIM :

To implement communication between a parent and child process using an anonymous pipe and measure the communication time.

Program:



```
alekhya@AlekhyaReddy: ~/OS
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int pipefd[2];
    pid_t pid;

    int data[5] = {10, 20, 30, 40, 50};

    struct timespec start, end;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* Start timer */
    clock_gettime(CLOCK_MONOTONIC, &start);

    /* Create child */
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    /* Parent - Producer */
    if (pid > 0)
    {
        close(pipefd[0]);

        printf("Producer (Parent) started\n");
        printf("Parent PID: %d\n\n", getpid());

        for (int i = 0; i < 5; i++)
```



alekhya@Alekhyareddy: ~/OS



GNU nano 7.2

```
    printf("Producer (Parent) started\n");
    printf("Parent PID: %d\n\n", getpid());

    for (int i = 0; i < 5; i++)
    {
        write(pipefd[1], &data[i], sizeof(data[i]));

        printf("Produced: %d\n", data[i]);
    }

    close(pipefd[1]);

    wait(NULL);

    /* Stop timer */
    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    printf("\nCommunication completed.\n");
    printf("Communication time: %.6f seconds\n", time_taken);

    printf("Producer completed.\n");
}

/* Child - Consumer */
else
{
    close(pipefd[1]);

    int value;

    printf("\nConsumer (Child) started\n");
    printf("Child PID: %d\n\n", getpid());

    while (read(pipefd[0], &value, sizeof(value)) > 0)
    {
        printf("Consumed: %d\n", value);
    }

    close(pipefd[0]);

    printf("\nConsumer completed.\n");

    exit(0);
}

return 0;
```

}

Help
 Exit

Write Out
 Read File

Where Is
 Replace

Cut
 Paste

Output:

```
alekhya@Alekhyareddy: ~/OS X + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ pwd
/home/alekhya/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano producer_consumer.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra producer_consumer.c -o producer_consumer
alekhya@Alekhyareddy:~/OSSP$ ./producer_consumer
Producer (Parent) started
Parent PID: 513

Produced: 10
Produced: 20
Produced: 30
Produced: 40
Produced: 50

Consumer (Child) started
Child PID: 514

Consumed: 10
Consumed: 20
Consumed: 30
Consumed: 40
Consumed: 50

Consumer completed.

Communication completed.
Communication time: 0.001161 seconds
Producer completed.
alekhya@Alekhyareddy:~/OSSP$ |
```

2 . Develop a program that executes the equivalent of `ls -l | grep ".c"` using System Calls

Program:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    /* First child - ls -l */
    if (pid1 == 0)
    {
        close(pipefd[0]);

        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        /* Execute ls -l */
        execlp("ls", "ls", "-l", (char *)NULL);

        perror("exec ls");
        exit(1);
    }
}
```



alekhya@Alekhyareddy: ~/OS



GNU nano 7.2

```
        perror("exec ls");
        exit(1);
    }

    /* Create second child */
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    /* Second child - grep ".c" */
    if (pid2 == 0)
    {
        close(pipefd[1]);

        /* Redirect stdin from pipe */
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        /* Execute grep ".c" */
        execlp("grep", "grep", ".c", (char *)NULL);

        perror("exec grep");
        exit(1);
    }

    /* Parent */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for both children */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nPipeline execution completed.\n");

    return 0;
}
```

Output:

```
alekhya@AlekhyaReddy:~/OSSP$ nano ls_grep.c
alekhya@AlekhyaReddy:~/OSSP$ gcc -Wall -Wextra ls_grep.c -o ls_grep
alekhya@AlekhyaReddy:~/OSSP$ ./ls_grep
drwxr-xr-x 2 alekhya alekhya 4096 Aug  1 09:08 docs
drwxr-xr-x 2 alekhya alekhya 4096 Aug  1 09:07 include
-rw-r--r-- 1 alekhya alekhya 1376 Sep 18 03:32 ls_grep.c
-rw-r--r-- 1 alekhya alekhya  420 Aug 13 11:45 main_loop.c
-rwxr-xr-x 1 alekhya alekhya 16488 Sep 18 03:26 producer_consumer
-rw-r--r-- 1 alekhya alekhya 1677 Sep 18 03:26 producer_consumer.c
drwxr-xr-x 2 alekhya alekhya 4096 Aug  1 09:08 scripts
drwxr-xr-x 2 alekhya alekhya 4096 Aug  4 05:32 src
-rw-r--r-- 1 alekhya alekhya 1851 Aug 17 13:42 task1_wait.c
-rw-r--r-- 1 alekhya alekhya 1851 Aug 17 13:46 task1_wait.c.save
-rw-r--r-- 1 alekhya alekhya 2380 Aug 13 11:56 task2_quotes.c
-rw-r--r-- 1 alekhya alekhya 4088 Aug 13 12:53 task3_double_quotes.c
-rw-r--r-- 1 alekhya alekhya  940 Aug 17 06:02 task3_fork.c
-rw-r--r-- 1 alekhya alekhya 1661 Aug  7 08:09 task3_input.c
-rwxr-xr-x 1 alekhya alekhya 16488 Aug 17 04:59 task4_escape
-rw-r--r-- 1 alekhya alekhya 2297 Aug 17 04:59 task4_escape.c
-rw-r--r-- 1 alekhya alekhya 3976 Aug  7 08:19 task4_history.c
-rw-r--r-- 1 alekhya alekhya  954 Aug 17 13:32 task4_states.c
-rw-r--r-- 1 alekhya alekhya 2612 Aug  7 08:30 task5_memory.c
-rwxr-xr-x 1 alekhya alekhya 16696 Aug 17 05:03 task5_process
-rw-r--r-- 1 alekhya alekhya 2092 Aug 17 05:03 task5_process.c
-rw-r--r-- 1 alekhya alekhya  662 Aug 17 13:53 week4_T2.c

Pipeline execution completed.
alekhya@AlekhyaReddy:~/OSSP$ |
```